

Informacioni sistemi – drugi deo projekta

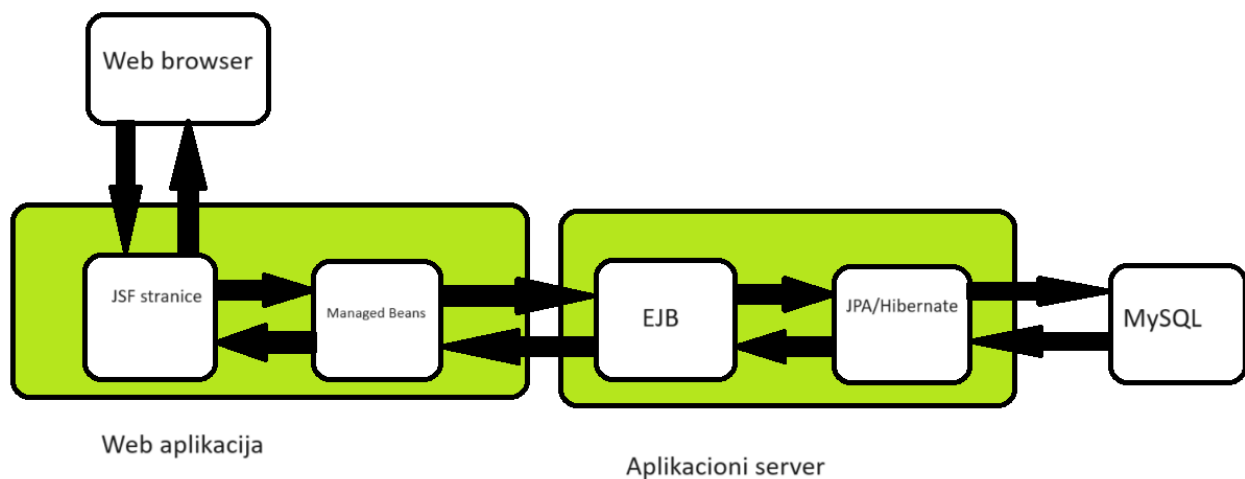
Ime i prezime: Lazar Trifunovic indeks: 18967

Zadatak:

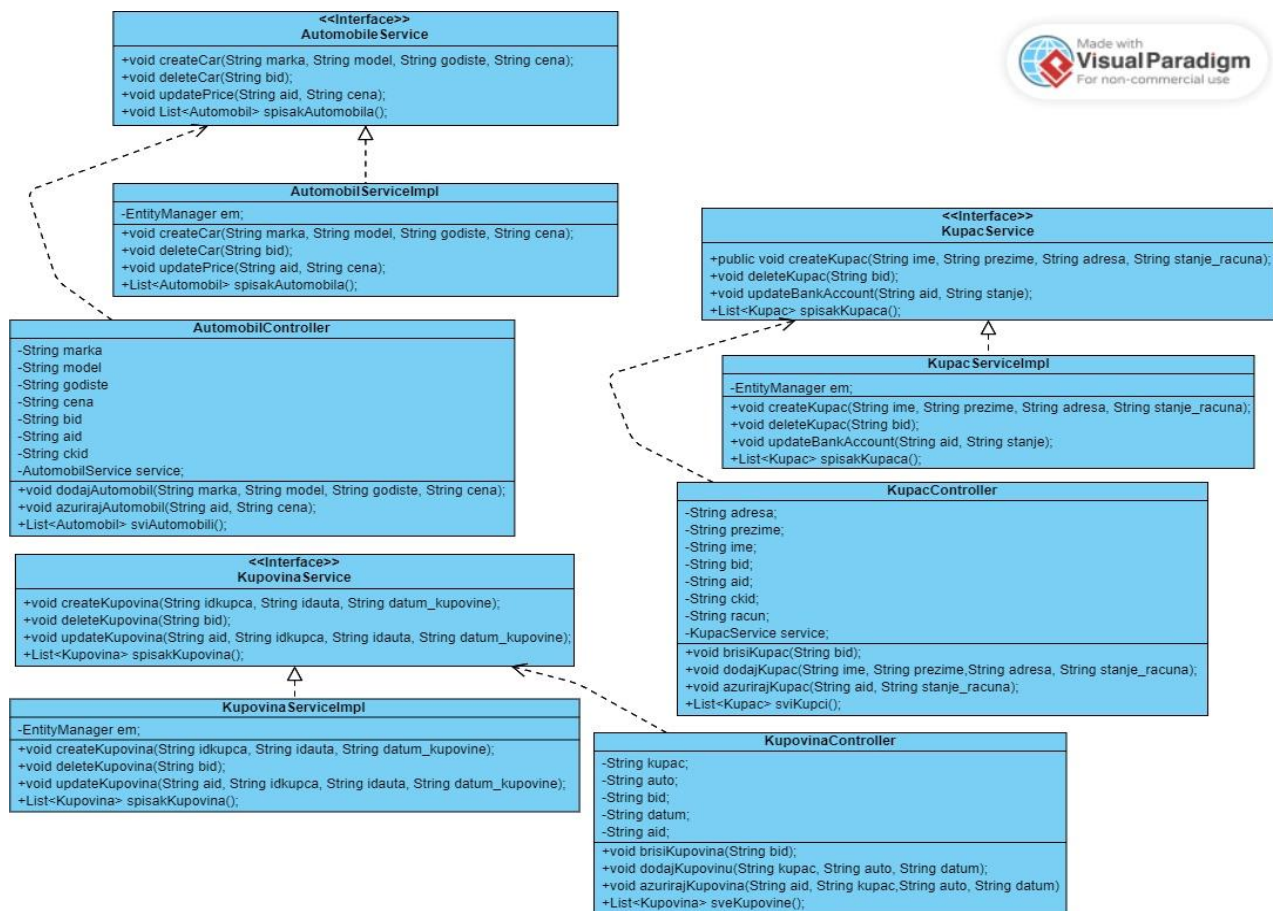
124. Informacioni sistem za prodaju automobila marke Volvo - današnjici, sve je češća pojava da proizvođači nude posebne pogodnosti svojim redovnim klijentima. Na taj način proizvođači organizuju klubove potrošača, koji olakšavaju klijentima pregled i aktiviranje pogodnosti.

Obrazac: tok akcije

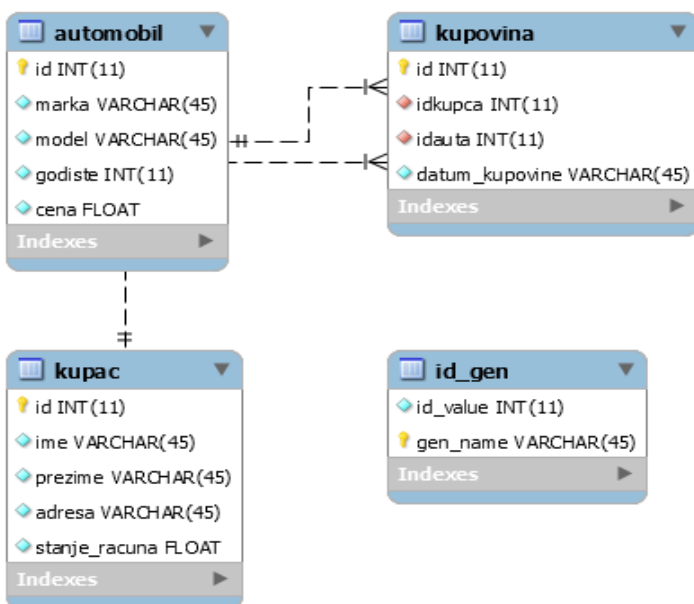
Arhitektura aplikacije:



UML klasni dijagram:

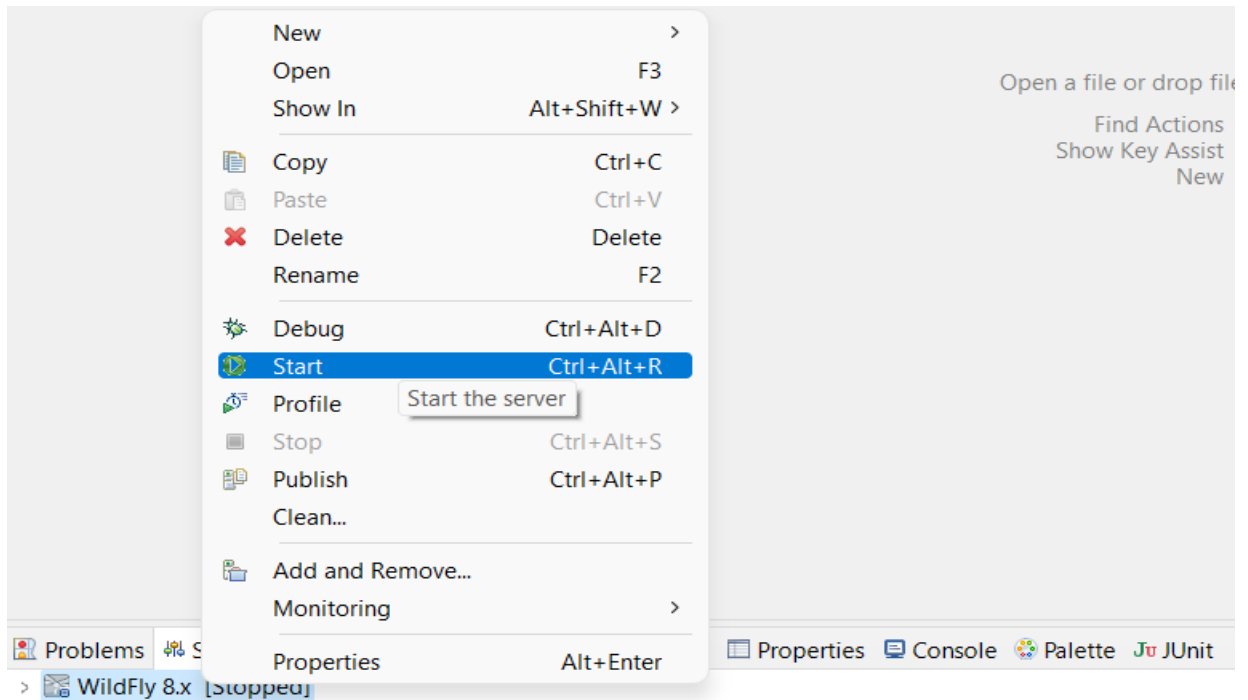


Sema baze podataka:

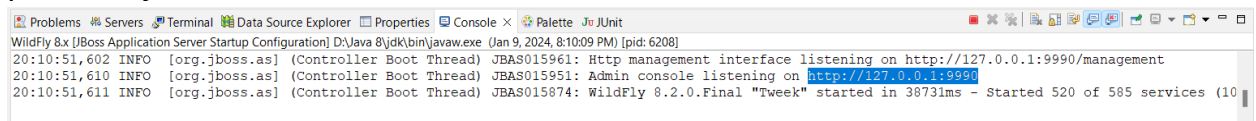


Deployment:

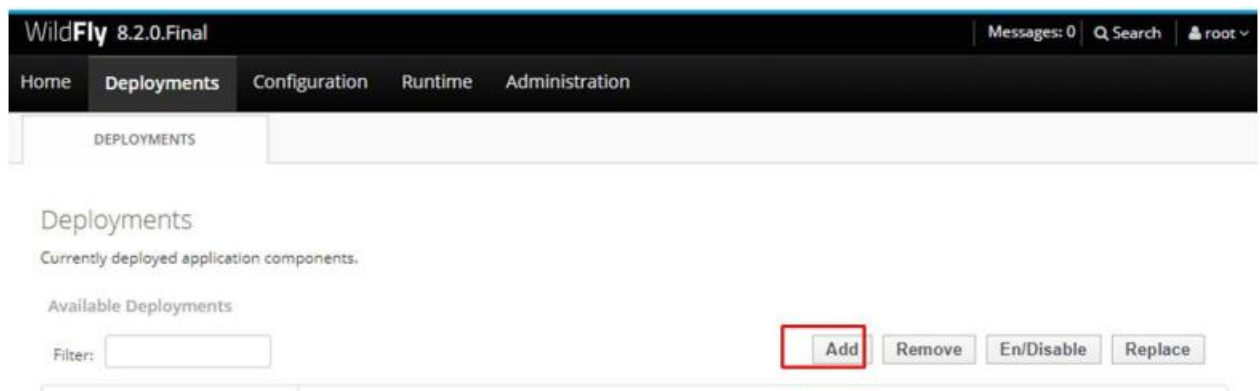
1. Startovati WildFly server – desni klik na „WildFly 8.x at localhost“ u Servers tabu Eclipse-a



2. Pristupiti mu iz browser-a na adresi koja se prikaže u konzoli nakon pokretanja servera



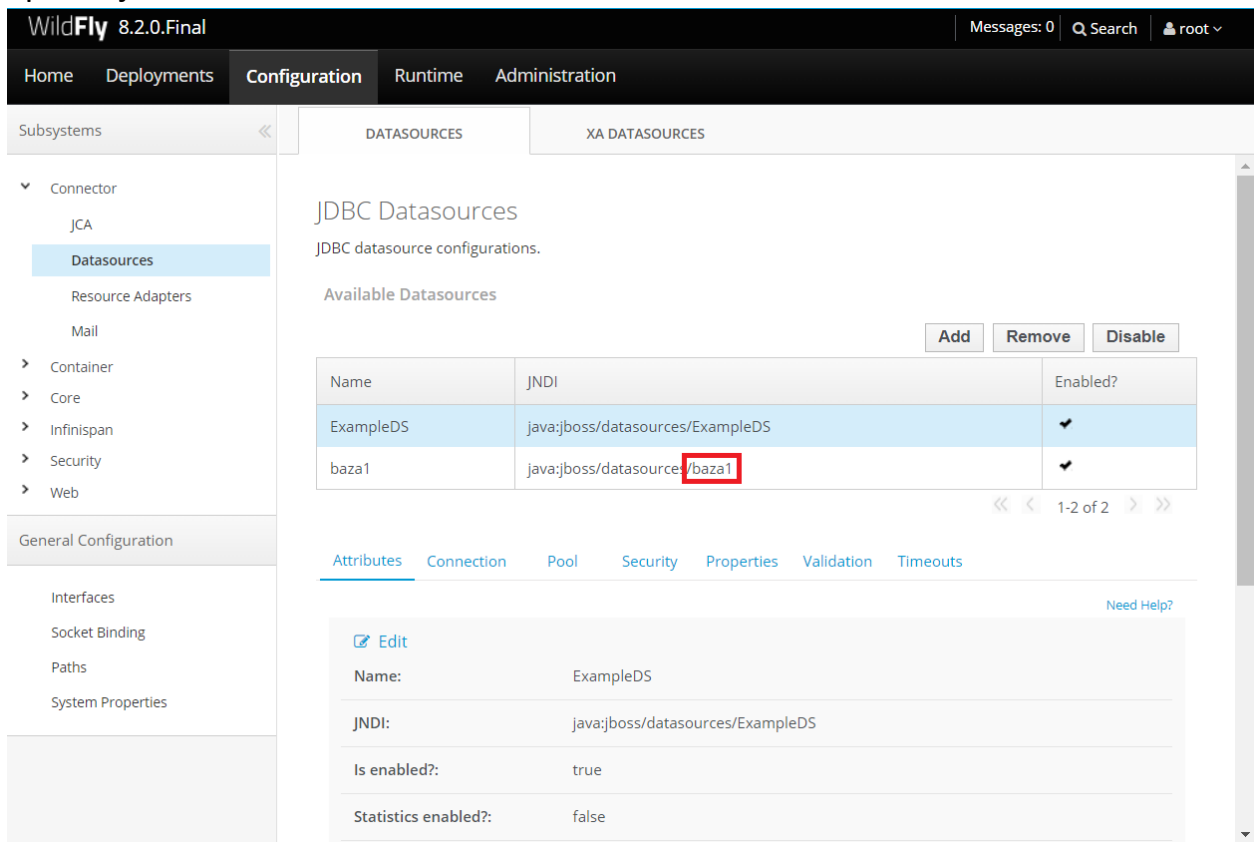
3. U kartici Deployments odabrati opciju Add



4. Selektovati odgovarajući jar fajl za mysql konektor iz fajl sistema i sačuvati taj Deployment



5. U kartici Configuration dodati stavku za bazu podataka koja se koristi u aplikaciji



Na slici je označen “/baza1” i taj deo mora odgovarati imenu šeme u bazi podataka. Zatim je *obavezno* testiranje konekcije.

Ukoliko je povezivanje uspešno, uspešno je uspostavljena konekcija sa bazom podataka

Za Maven:

DEPLOYMENTS

Deployments

Currently deployed application components.

Available Deployments

Filter:

Add

Remove

En/Disable

Replace

CarCompanyApp.war



ejb3

AutomobilServiceImpl

mysql-connector-java-5.1. ...



jpa

KupovinaServiceImpl

projekat1-0.0.1-SNAPSHOT.jar



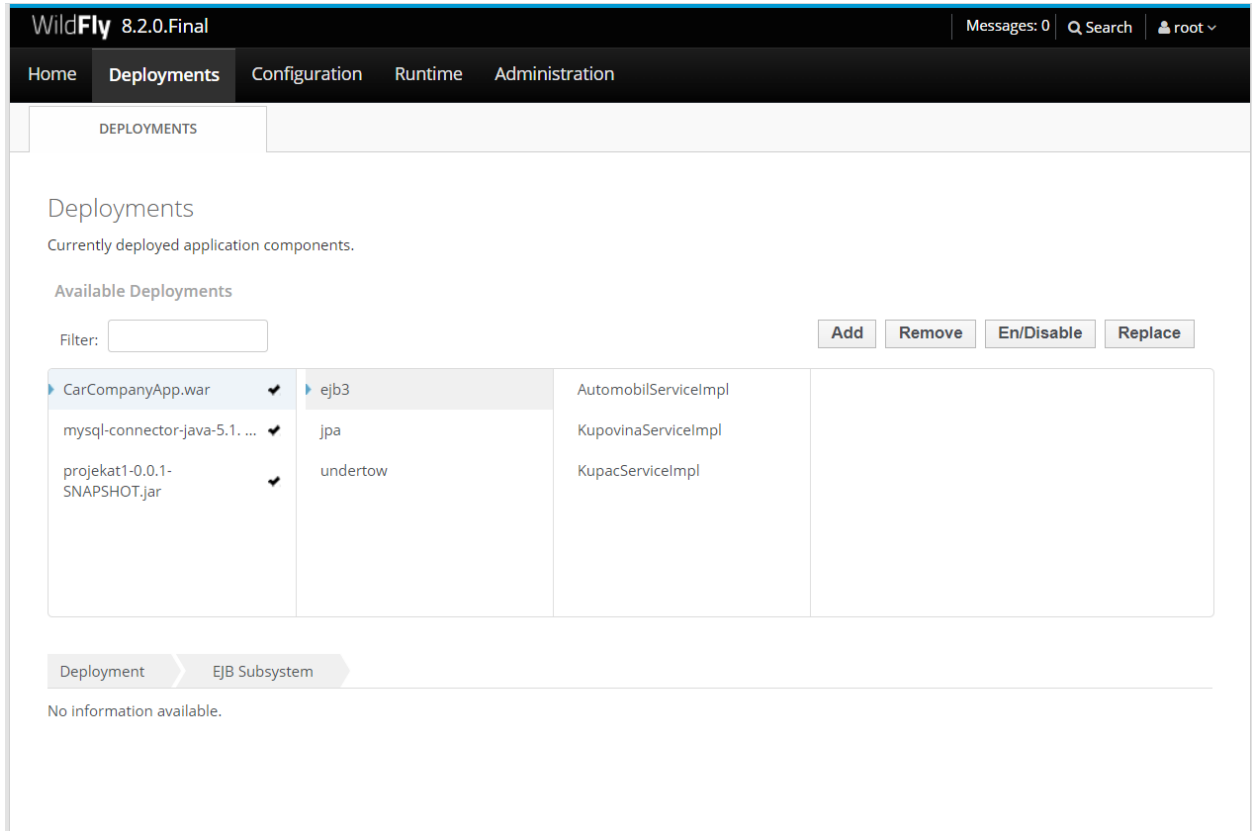
KupacServiceImpl

Deployment

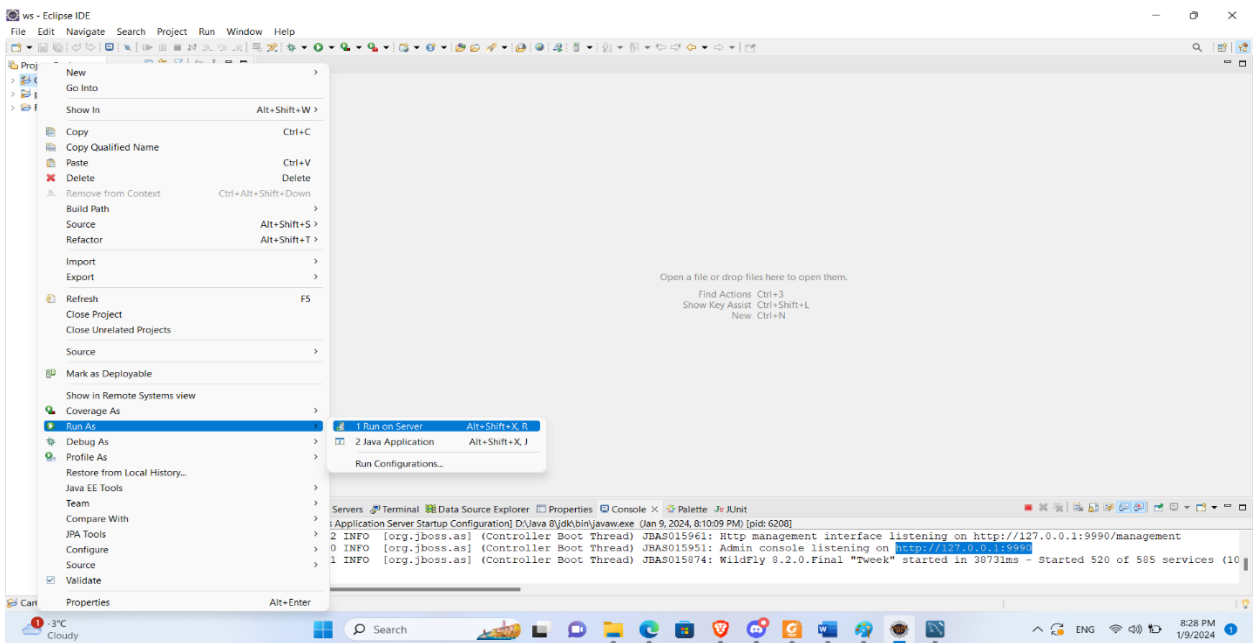
EJB Subsystem

No information available.

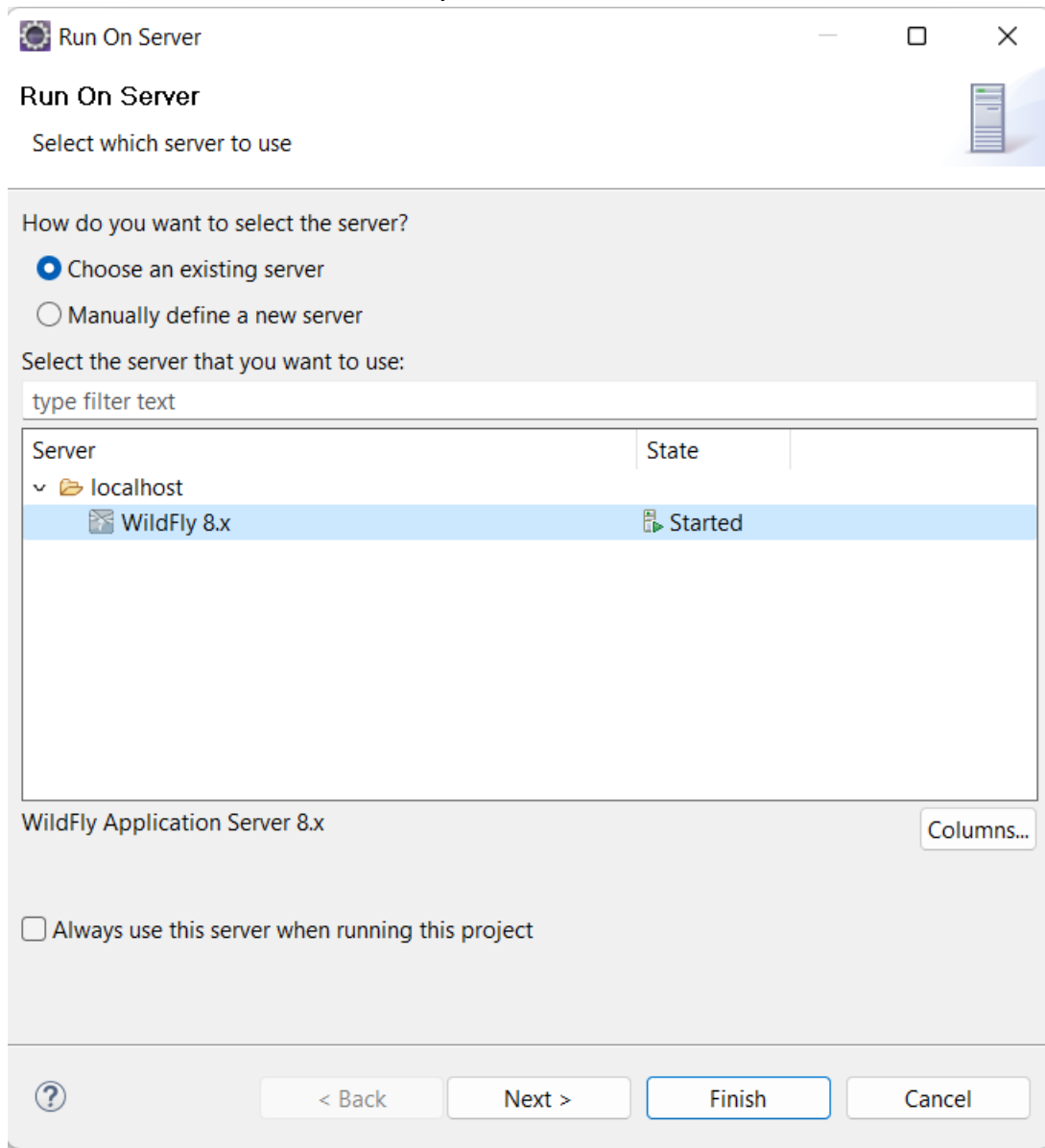
Za Web:



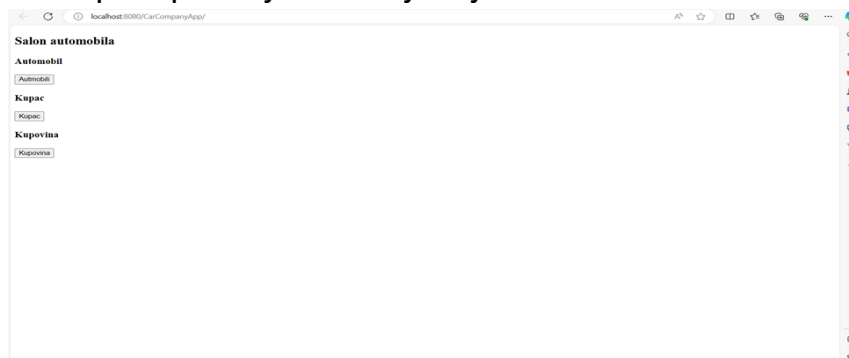
6. Pokrenuti izvršenje aplikacije desnim klikom na projekat u Eclipse-u -> Run As -> Run on Server



7. Odabrati već startovan WildFly Server



8. Pristupiti aplikaciji na dobijenoj adresi



Adresa: <localhost:8080/CarCompanyApp/index.xhtml>

TEST CASE:

1. Test funkcije za unos automobila

Naziv testa: TestCase1

Testiramo da li se automobili ispravno unose u bazu, u bazi ne smeju postojati dva automobila sa istim nazivom modela, jer za tim nema potrebe, na kraju se test primer brise iz baze

Preduslovi: Ne postoji automobil istog modela u bazi

Koraci testa:

1. Dodati nekoliko automobila u bazu
2. Pozvati funkciju za unos automobila u bazu
3. Proveriti da li je funkcija vratila vrednost razlicitu od NULL
4. Obrisati uneti test podatak

Očekivani rezultat:

Povratna vrednost koja nije NULL, unet i obrisani podatak iz baze

Post-uslov:

U bazi postoji model

Dobijeni rezultati:

Povratna vrednost funkcije za unos automobila u bazu vraća vrednost koja nije NULL.

Status:

Uspešan.

2. Test funkcije za azuriranje racuna kupca

Naziv testa: TestCase2

Testiramo da li se stanje na racunu kupca uspesno update-uje

Preduslovi: Ne postoji

Koraci testa:

1. Poslati id zeljenog kupca i novo stanje racuna
2. Pozvati funkciju za azuriranje stanja na racunu
3. Proveriti da li je funkcija postavila stanje racuna na zeljenu vrednost
4. Vratiti stanje racuna na prethodnu vrednost

Očekivani rezultat:

Povratna vrednost novo stanje racuna

Post-uslov:

Vracamo prethodno stanje racuna

Dobijeni rezultati:

Stanje na racunu je azurirano na novu vrednost, a nakon toga vraceno na staru

Status:

Uspešan.

3. Test funkcije za azuriranje racuna kupca

Naziv testa: TestCase3

Testiramo da li je kupac sa id=1 izvrrio kupovinu u salonu automobile

Preduslovi: Kupac mora da postoji u bazi

Koraci testa:

1. Dodati kupca u bazu
2. Pozvati funkciju za proveru da li je kupac izvrrio kupovinu
3. Proveriti da li je povratna vrednost funkcije razlicita od NULL, ukoliko jeste, postoji kupovina

Očekivani rezultat:

Povratna vrednost razlicita od NULL

Post-uslov:

Ne postoji

Dobijeni rezultati:

Povratna vrednost razlicita od NULL, postoji kupovina

Status:

Uspešan.